

GRID-X

Department: Computer Science & Artificial Intelligence

Team Members:

Syed Ibrahim Ali (1604-22-748-010)

Faiz Ahmed (1604-22-748-013)

Husamuddin Bilal (1604-22-748-017)

Guide: Uma N Dulhare

College: Muffakham Jah College of Engineering & Technology



ABSTRACT

- GRID-X is an end-to-end machine learning system that predicts lap times, race outcomes, driver styles, and optimal pit strategies for Formula 1.
- Combines supervised, unsupervised, reinforcement, ensemble, transfer learning and computer vision paradigms.
- Uses powerful Supervised learning models to predict lap times, and race outcomes and unsupervised learning to group drivers according to driving style.
- Uses more advanced ml paradigms like reinforcement learning to optimise race strategy, machine vision to analyse circuits and transfer learning to ease training.
- SHAP explainability provides feature-level transparency for lap-time predictions.
- All models are served through a unified FastAPI backend, ready for frontend integration.

EXISTING SYSTEM

- Existing F1 prediction tools are often single-purpose (e.g., only lap-time or only race outcome) and lack transparency.
- Commercial platforms (like some sports betting sites) use basic statistical models or single ML models without multi-paradigm integration.
- No publicly available system combines regression, classification, clustering, reinforcement learning, and computer vision in one platform.
- Most solutions do not provide explainability or strategy optimization; they are “black boxes.”
- Our system addresses these gaps with a comprehensive, multi-paradigm, explainable AI platform.

LITERATURE SURVEY

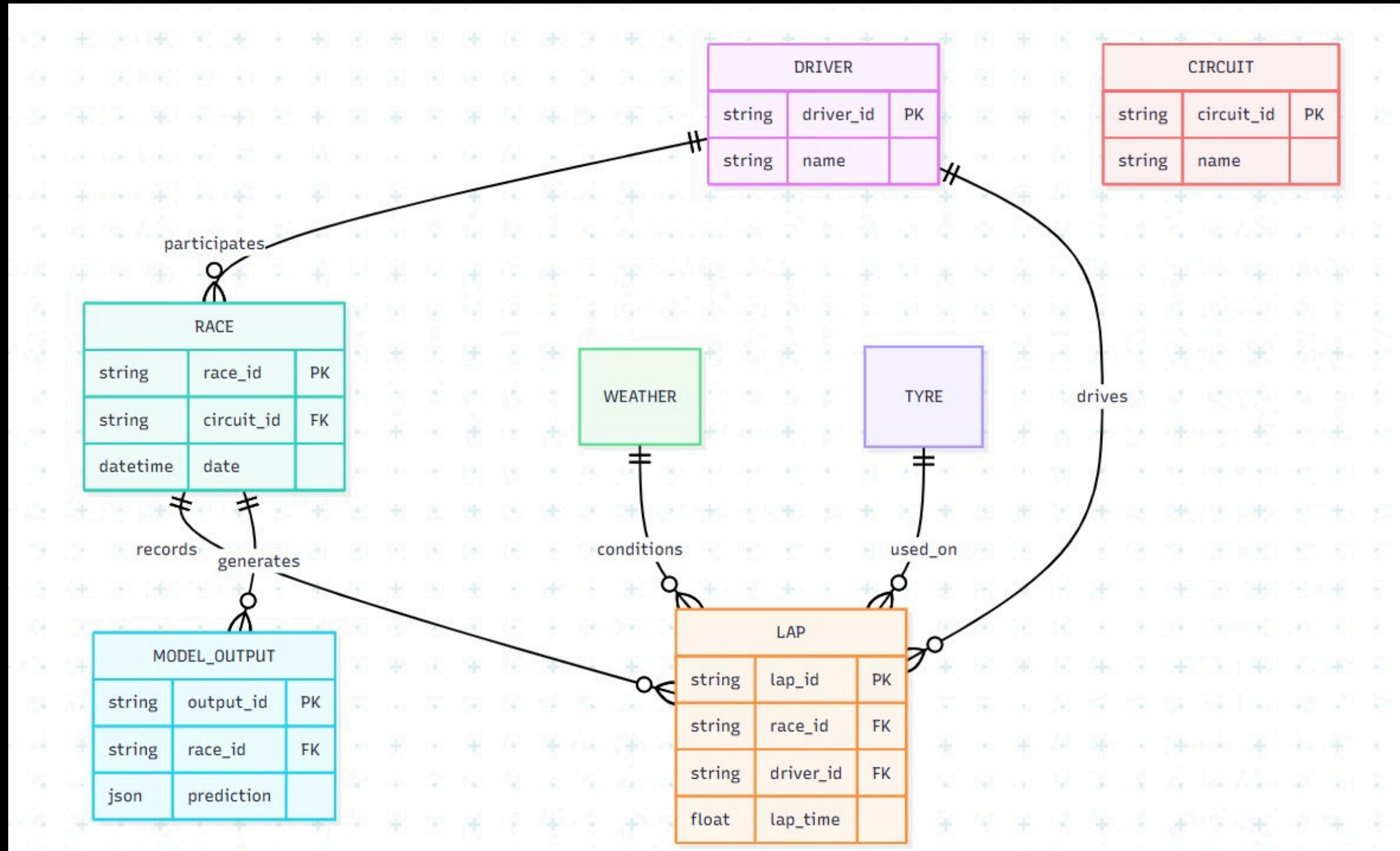
Paper Title	Key Findings	Shortcomings
Deep Neural Network-based Lap Time Forecasting of Formula 1 Racing	DNN predicts qualifying lap times accurately	Ignores race strategy, tyre wear, explainability
Optimizing Pit Stop Strategies with Competition in a Zero-Sum Feedback Stackelberg Game in Formula 1	Game-theory model for competitive pit strategies	Limited to two drivers, high complexity
Exploring Reasons of Formula 1 Team's Underperforming in F1 Racing	Identifies financial & technical causes of poor performance	No quantitative or predictive modelling
Design, Analysis and Fabrication of F1 Car for Improving Aerodynamic Efficiency	CFD & wind tunnel improve aerodynamic efficiency	No real race data integration
Design, Analysis and Fabrication of F1 Car for Improving Aerodynamic Efficiency	Highlights impact of tyre temperature & wear modelling	Not linked to ML or strategy optimization

PROPOSED SYSTEM

System Architecture Diagram

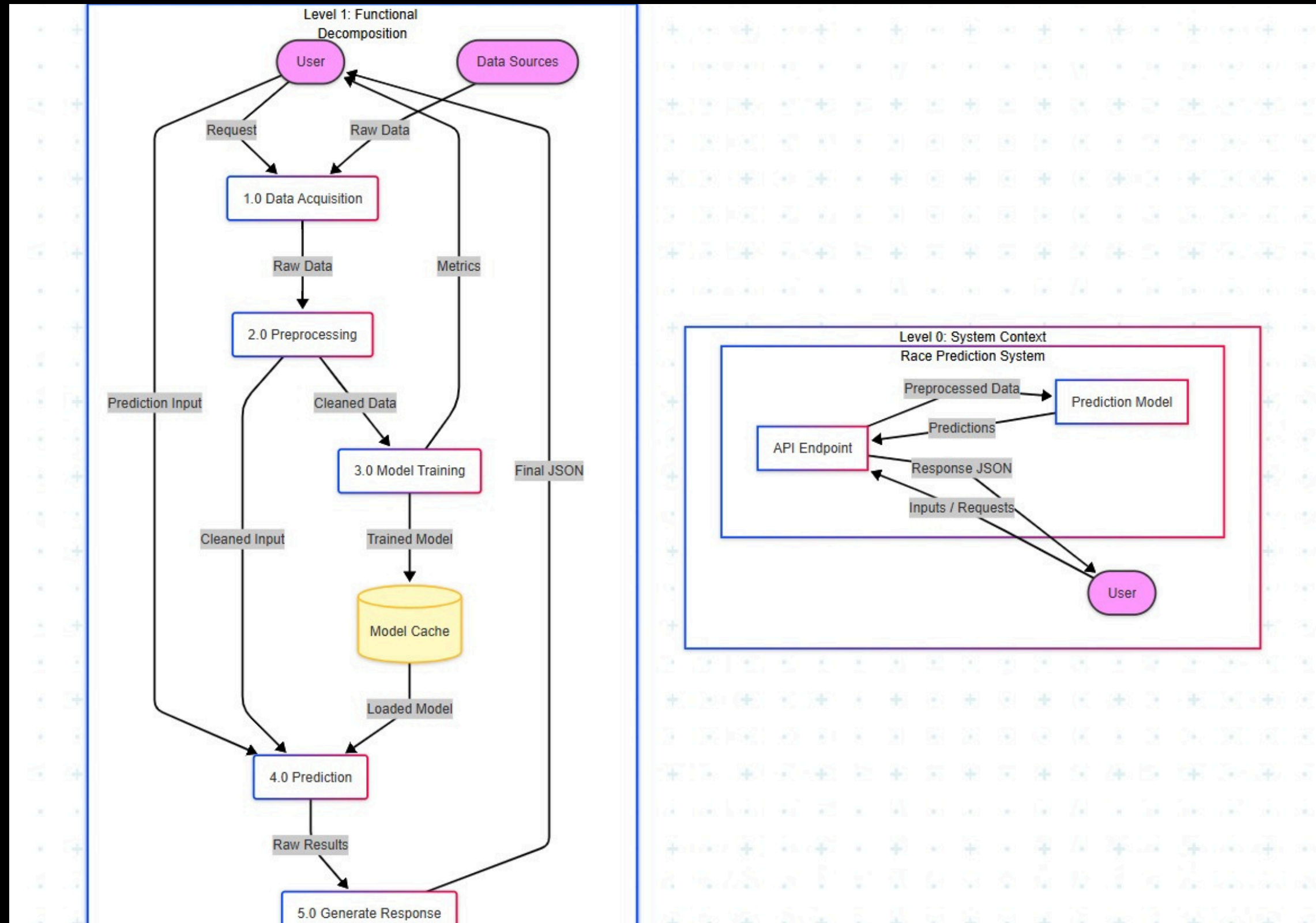
- Data Layer: FastF1 API (modern telemetry) + Kaggle historical dataset (1950–2020) → preprocessing pipelines.
- Model Layer:
 - Lap-Time Predictor (RandomForest)
 - Driver Style Analyzer (KMeans)
 - Race Outcome Classifier (Dual-era XGBoost + Logistic Ensemble)
 - Pace Forecaster (LSTM)
 - Strategy Optimizer (DQN)
 - Circuit Analyzer (ResNet50 CNN)
- Integration Layer: `int_en_pred_2.py` loads all models; provides unified methods.
- API Layer: FastAPI server with six endpoints (`/predict`, `/stint-simulate`, `/next-lap`, `/strategy-optimize`, `/analyze-circuit`, `/explain-lap`).
- Frontend: (Planned) React dashboard

SYSTEM ARCHITECTURE DESIGN



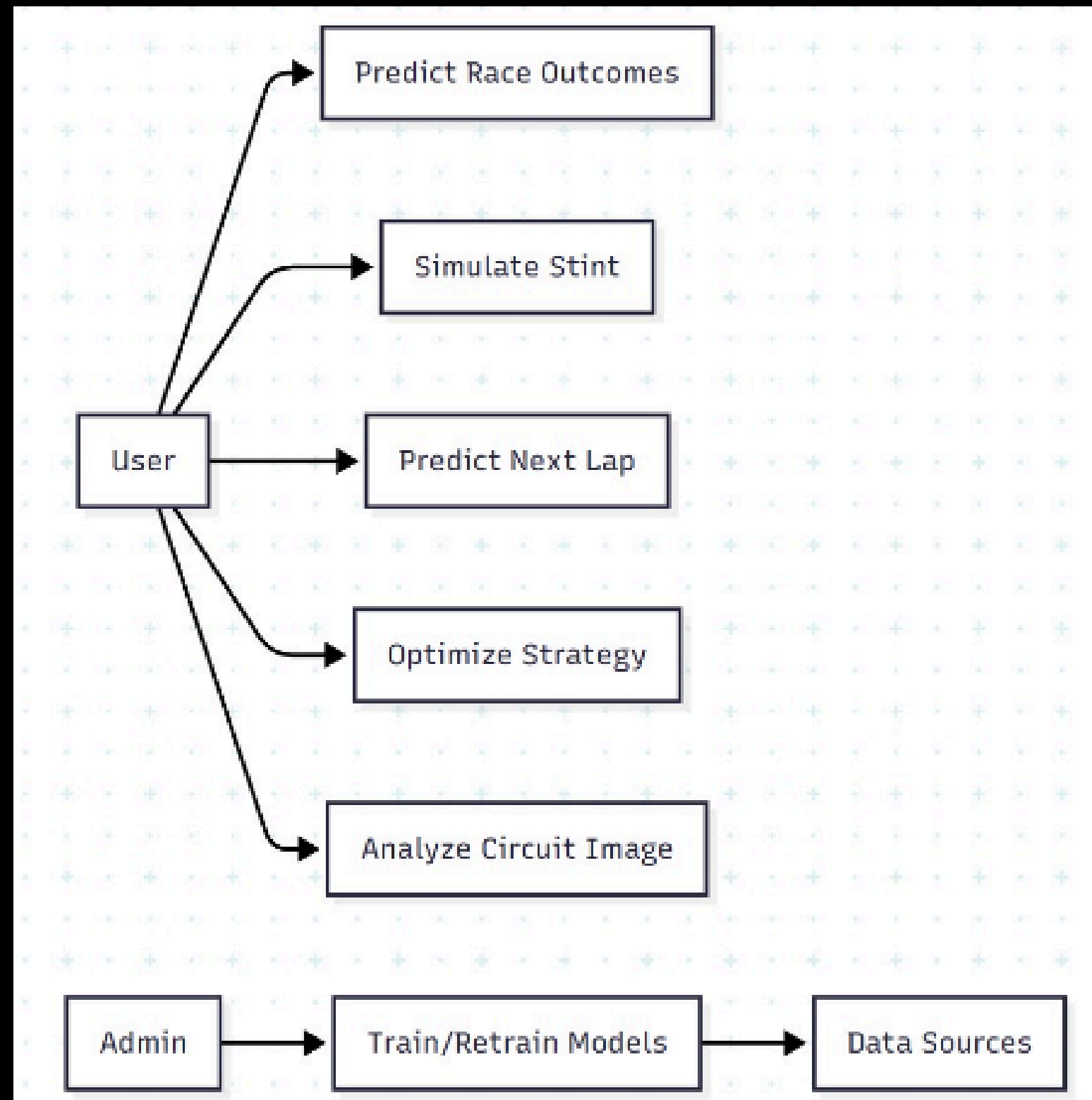
E-R Diagram

SYSTEM ARCHITECTURE DESIGN



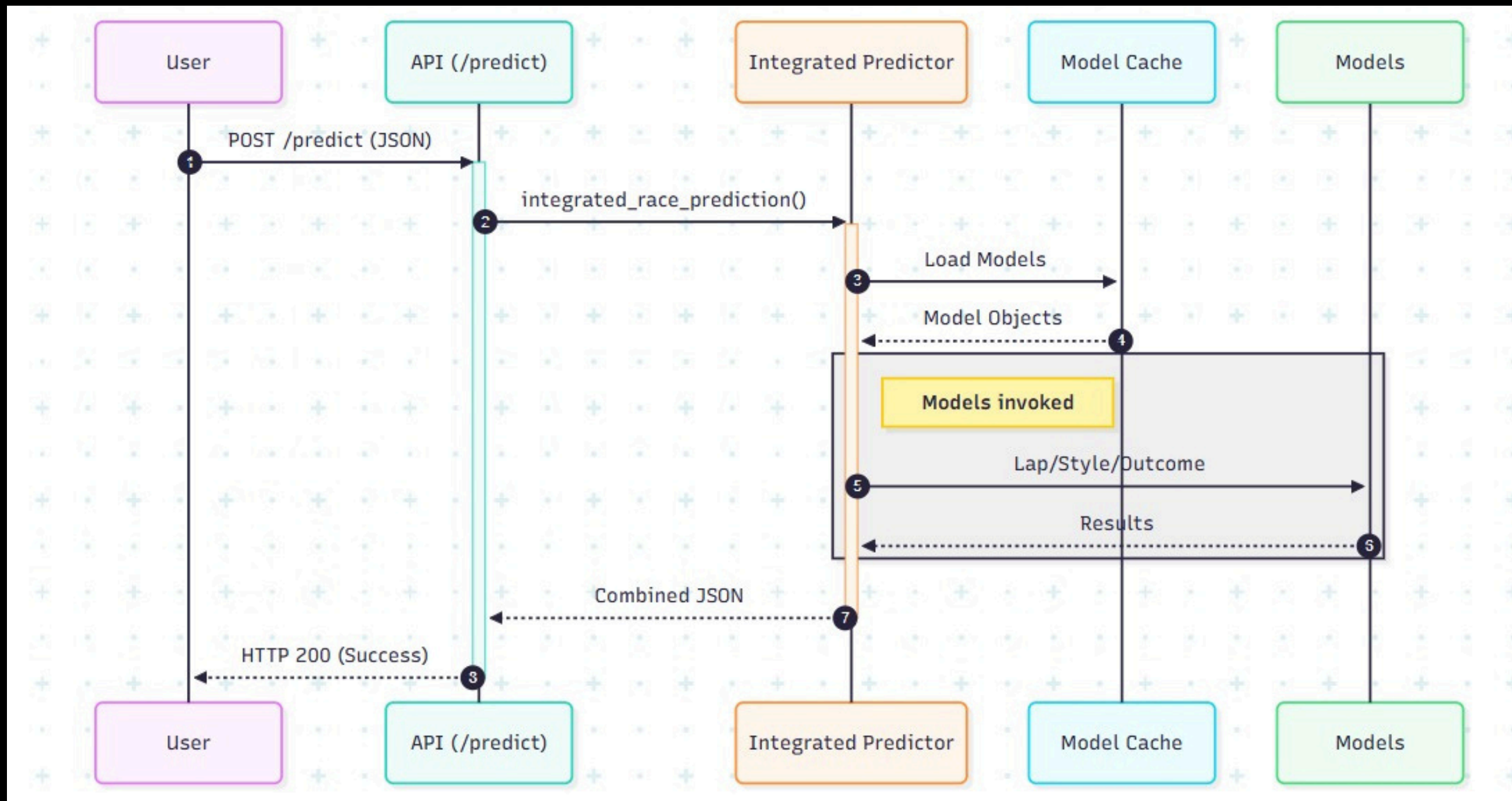
Data Flow Dagrams

UML DIAGRAMS



Use Case Diagram

UML DIAGRAMS



Sequence Diagram

METHODOLOGY

1. Lap-Time Prediction (Random Forest)

- Input: 21 features (weather, tyre age, stint progress, circuit, driver...)
- Output: Lap time (seconds)
- Algorithm: Random Forest Regressor with 100 trees; hyperparameter tuning via GridSearch.
- Performance: RMSE = 2.787s, $R^2 = 0.993$, MAE = 0.96s.

2. Race Outcome Classification (Dual-era Ensemble)

- Historical model: XGBoost on 1950–2020 data (13 features).
- Modern model: XGBoost on 2021–2024 data (10 features).
- Ensemble: Logistic regression on the two model probabilities.
- Accuracy: Win 96.4%, Podium 93.2%, Points 88.6%.

3. Driver Style Clustering (KMeans)

- Features engineered from telemetry (consistency, aggression, braking, etc.).
- Optimal $k=4$ (Silhouette score).
- Resulting clusters: Aggressive, Smooth, Opportunistic, Balanced.

4. Reinforcement Learning (DQN)

- Environment: Custom RaceEnv with linear tyre degradation (0.1–0.5 s/lap, compound-specific).
- State: (lap norm, tyre age norm, one-hot compound).
- Actions: Stay out / pit to SOFT / MEDIUM / HARD.
- Reward: Negative lap time (minimize total race time).
- Training: DQN with experience replay, target network, 1000 episodes.
- Result: Learns optimal pit stops (e.g., 4 stops for 78-lap race).

5. Circuit Recognition (CNN)

- Model: ResNet50 pre-trained on ImageNet, fine-tuned on 24 circuit schematic images.
- Data augmentation: rotation, flip, color jitter.
- Output: Top-3 circuit predictions with confidence; embeddings for similarity search.

6. Explainability (SHAP)

- TreeExplainer on RandomForest lap-time model.
- For a given prediction, returns top-10 features with SHAP values (positive/negative impact).

DATASET DESCRIPTION

Modern Data (2021–2024)

- Source: FastF1 API
- Records: 90,901 clean laps
- Features: 57 columns including lap times, sector times, speeds, weather, tyre compound, stint, driver, team.
- Drivers: 34
- Events: 29 race weekends

Historical Data (1950–2020)

- Source: Kaggle F1 Dataset
- Records: 26,759 race entries
- Features: driver, constructor, grid, position, points, year, circuit, etc.
- Drivers: 861
- Constructors: 211
- Circuits: 77

Circuit Image Dataset

- Source: Web scraping + teammate collection
- Images: 24 schematic circuit maps (PNG, 224×224, white background, black track lines)
- Metadata: circuit name, full name, location, track type, length, corners, first GP, lap record, etc.

Module Implementation

Code Snippets

Lap-Time Predictor

```
def train_models(self, X, y):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
    model = RandomForestRegressor(n_estimators=100, random_state=42)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print(f"RMSE: {np.sqrt(mean_squared_error(y_test, y_pred)):.3f}s")
    return model
```

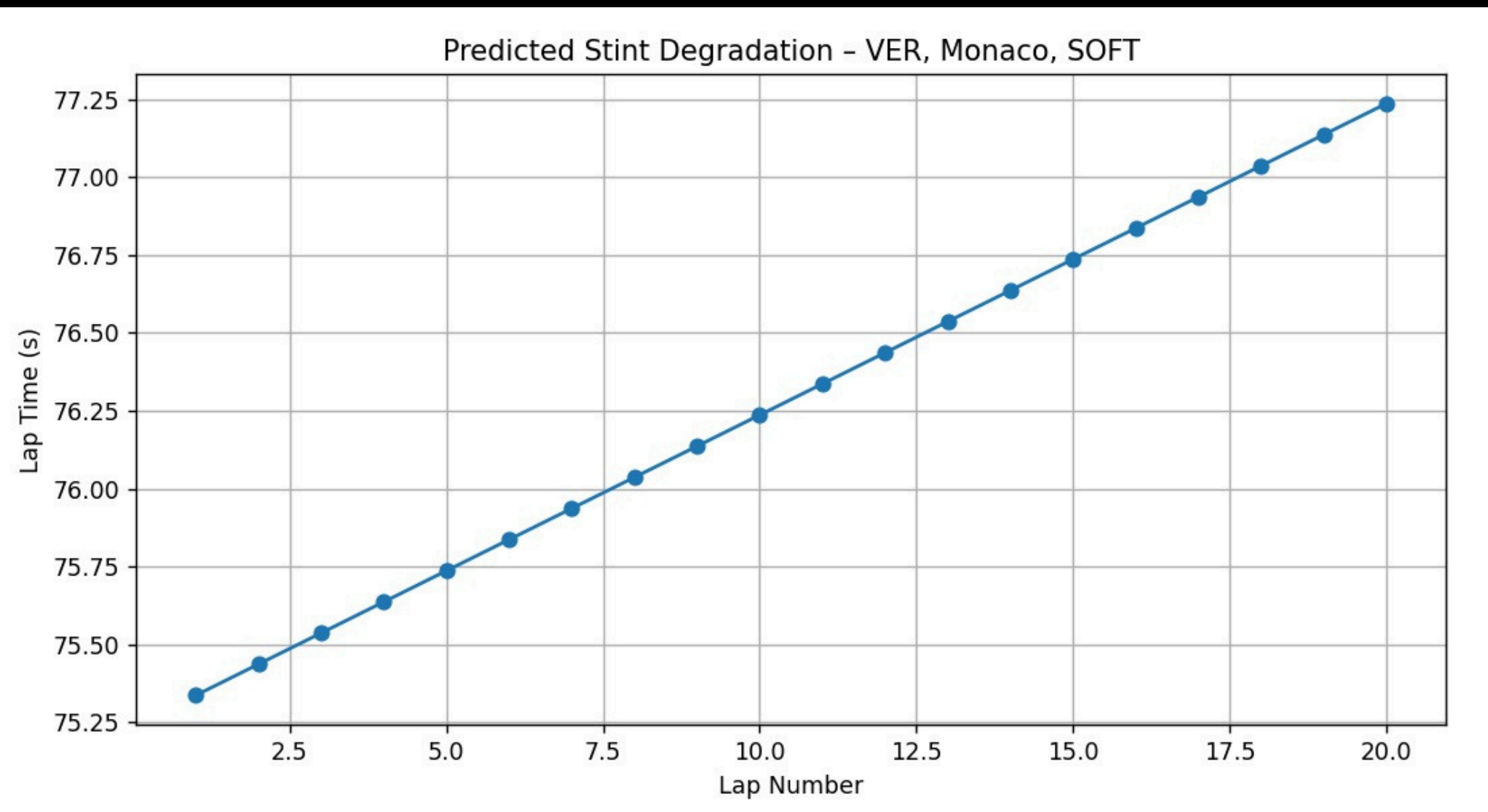
DQN Agent – Action Selection

```
def act(self, state, eval_mode=False):
    if not eval_mode and np.random.rand() <= self.epsilon:
        return np.random.randint(self.action_size)
    state_t = torch.FloatTensor(state).unsqueeze(0).to(self.device)
    with torch.no_grad():
        q_values = self.policy_net(state_t)
    return q_values.argmax().item()
```

```
def get_model(num_classes):  
    model = models.resnet50(pretrained=True)  
    for param in model.parameters():  
        param.requires_grad = False  
    num_features = model.fc.in_features  
    model.fc = nn.Linear(num_features, num_classes)  
    return model
```

CNN model definition

RESULTS



Stint Degradation

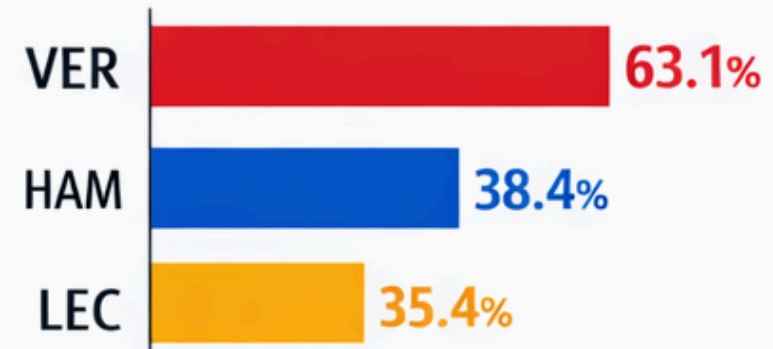
Lap-Time Prediction (Random Forest)



Absolute Error: 0.036s

Model RMSE: 2.787s

Race Outcome Prediction



Driver Style Analysis (VER)



Cluster: **OPPORTUNISTIC**

Optimal Pit Strategy (78-Lap Race)



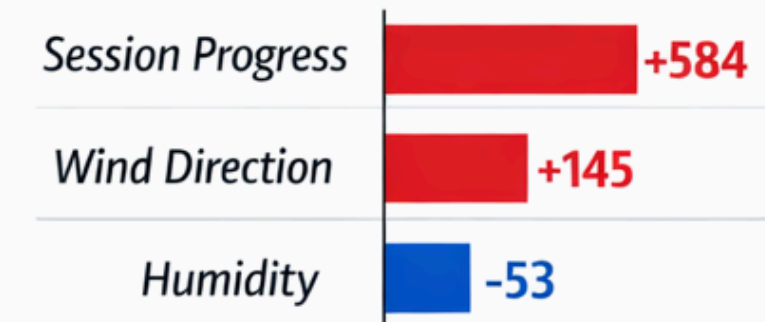
Total Race Time: 6025.24s

Circuit Recognition



1. Monaco (0.92)
2. Singapore (0.87)
3. Baku (0.76)

SHAP Explainability



Positive = Slower

Negative = Faster



THANK

YOU

